

Universidade Federal
de Campina Grande



UNIVERSIDADE FEDERAL DE CAMPINA GRANDE
DEPARTAMENTO DE ENGENHARIA ELÉTRICA E INFORMÁTICA
DISCIPLINA: CAPACITAÇÃO EM PROCESSAMENTO DIGITAL DE SINAIS

RELATÓRIO PRÁTICO

Relatório 2: Transformadas Discretas

Aluno: Guilherme José Mendes Jerônimo
Professor: Prof. Dr. Edson Porto da Silva
Semestre: 2026.2

Campina Grande - PB
2026

Sumário

1 Resumo **1**

2 Introdução **1**
2.1 Transformada Discreta de Fourier (DFT) e Suas Propriedades 1
2.2 Convolução Circular vs. Convolução Linear 1
2.3 Convolução de Sinais Longos: Overlap-and-Add e Overlap-and-Save 2
2.4 Algoritmo da Transformada Rápida de Fourier (FFT) 2
2.5 Objetivos do Trabalho 3

3 Desenvolvimento **3**
3.1 Implementação da DFT/IDFT e Validação de Propriedades 3
3.2 Convolução Linear via Convolução Circular 4
3.3 Implementação dos Métodos Overlap-and-Add e Overlap-and-Save 4
3.4 Implementação da FFT (Radix-2) e Análise de Desempenho 4

4 Resultados e Análise **5**
4.1 Aplicação do Algoritmo da DFT e a Reconstrução Temporal da IDFT . . . 5
4.2 Validação Experimental das Propriedades da DFT 6
4.3 Implementar a convolução linear a partir da convolução circular utilizando a DFT/IDFT. 7
4.3.1 A ideia central 7
4.3.2 Sinais de teste 7
4.3.3 O que foi implementado 8
4.3.4 Resultados 8
4.3.5 Por que a circular difere 8
4.3.6 Do circular para o linear 8
4.3.7 O que mudar no código 9
4.4 Implementar os métodos overlap-and-add e overlap and save para o cálculo de convoluções lineares via DFT/IDFT 9
4.4.1 Motivação 9
4.4.2 Visão geral dos dois métodos 9
4.4.3 Overlap-and-add passo a passo 10
4.4.4 Overlap-and-save passo a passo 10
4.4.5 Uso das funções 10
4.4.6 Sinais e parâmetros 10
4.4.7 Overlap-and-add 11
4.4.8 Overlap-and-save 11
4.4.9 Resultados dos Testes 11
4.4.10 Teste 1: sinal de 8 amostras, passo a passo 11
4.4.11 Teste 2: sinal de 100 amostras e vários tamanhos de bloco 12
4.4.12 Análise 13
4.4.13 Observações 13
4.5 Aplicação da FFT e Comparação Numérica com a DFT 13
4.6 Comparação de Desempenho com CuPy (GPU) 14
4.6.1 Objetivo 14
4.6.2 Ambiente e metodologia 15
4.6.3 Implementação em GPU 15

4.6.4 Validação numérica 15
4.6.5 Resultados 16
4.6.6 Discussão 17

5 Conclusão **19**

1 Resumo

Este relatório apresenta o estudo e a implementação da Transformada Discreta de Fourier (DFT) e da sua inversa (IDFT) em sinais unidimensionais, juntamente com a validação experimental de suas propriedades fundamentais. Investigou-se o cálculo da convolução linear obtida a partir da convolução circular no domínio da frequência, expandindo a análise para o processamento de sinais longos por meio das técnicas de blocagem Overlap-and-Add e Overlap-and-Save. Por fim, implementou-se o algoritmo da Transformada Rápida de Fourier (FFT), realizando uma avaliação comparativa de tempo de execução e exatidão dos resultados em relação à DFT direta. Os resultados demonstram a relevância da eficiência algorítmica no domínio da frequência para aplicações de Processamento Digital de Sinais (PDS).

O código completo e os experimentos estão disponíveis no repositório do projeto.

2 Introdução

No Processamento Digital de Sinais (PDS), a análise e o processamento de sequências numéricas discretas $x[n]$, com $n \in \mathbb{Z}$, no domínio da frequência são essenciais para o projeto de filtros e representação de sistemas Lineares e Invariantes no Tempo (LIT). A ferramenta fundamental para a análise espectral de sinais discretos de duração finita é a Transformada Discreta de Fourier (DFT), acompanhada de sua inversa (IDFT).

2.1 Transformada Discreta de Fourier (DFT) e Suas Propriedades

Para uma sequência $x[n]$ de comprimento N, definem-se a DFT direta $X[k]$ e a IDFT $x[n]$ pelas equações:

$$X[k] = \mathcal{F}\{x[n]\} = \sum_{n=0}^{N-1} x[n] \cdot W_N^{kn}, \quad k = 0, 1, \ldots, N-1 \tag{1}$$

$$x[n] = \mathcal{F}^{-1}\{X[k]\} = \frac{1}{N} \sum_{k=0}^{N-1} X[k] \cdot W_N^{-kn}, \quad n = 0, 1, \ldots, N-1 \tag{2}$$

onde $W_N = e^{-j\frac{2\pi}{N}}$ representa o fator de rotação (*twiddle factor*). A validação das propriedades operacionais da DFT — como linearidade, deslocamento temporal/frequencial, simetria conjugada e o Teorema de Parseval ($\sum |x[n]|^2 = \frac{1}{N} \sum |X[k]|^2$) — fundamenta a manipulação de sinais no domínio frequencial.

2.2 Convolução Circular vs. Convolução Linear

A filtragem de um sinal $x[n]$ por um sistema com resposta ao impulso $h[n]$ equivale à convolução no tempo. No entanto, a multiplicação direta de duas DFTs no domínio da frequência resulta na convolução circular (periódica), denotada por $\circledast$:

$$y_{\text{circ}}[n] = x[n] \circledast h[n] = \sum_{m=0}^{N-1} x[m] \cdot h[(n-m) \pmod N], \quad n = 0, 1, \ldots, N-1 \tag{3}$$

A relação no domínio da frequência é dada por:

$$\mathcal{F}\{x[n] \circledast h[n]\} = X[k] \cdot H[k] \tag{4}$$

Para obter a convolução linear tradicional (de comprimento $L = N_1 + N_2 - 1$) a partir da convolução circular entre $x[n]$ (tamanho N_1) e $h[n]$ (tamanho N_2), utiliza-se o preenchimento com zeros (*zero-padding*). Ambas as sequências são estendidas com zeros para um comprimento comum $N \geq N_1 + N_2 - 1$, permitindo calcular a saída por:

$$y[n] = (x * h)[n] = \mathcal{F}^{-1}\Big\{\mathcal{F}\{x_{\text{pad}}[n]\} \cdot \mathcal{F}\{h_{\text{pad}}[n]\}\Big\} \tag{5}$$

2.3 Convolução de Sinais Longos: Overlap-and-Add e Overlap-and-Save

Quando o sinal de entrada $x[n]$ é extremamente longo ou transmitido em tempo real ($N_1 \gg N_2$), o cálculo da DFT global torna-se computacionalmente inviável devido à alta latência e ao uso de memória. Nessas situações, o sinal é segmentado em blocos de tamanho L_B e processado por técnicas de filtragem em bloco:

- **Método Overlap-and-Add (OLA):** O sinal é dividido em blocos não sobrepostos $x_m[n]$ de tamanho L_B. Cada bloco recebe *zero-padding* até o tamanho $N = L_B + N_2 - 1$. A convolução linear parcial de cada bloco é calculada via DFT/IDFT e as saídas são somadas com sobreposição de $N_2 - 1$ amostras nas extremidades:

$$y[n] = \sum_m y_m[n - m \cdot L_B], \quad \text{onde } y_m[n] = \mathcal{F}^{-1}\big\{\mathcal{F}\{x_m[n]\} \cdot \mathcal{F}\{h_{\text{pad}}[n]\}\big\} \tag{6}$$

- **Método Overlap-and-Save (OLS):** O sinal é dividido em blocos sobrepostos de tamanho $N = L_B + N_2 - 1$, em que os primeiros $N_2 - 1$ pontos de cada bloco correspondem aos últimos $N_2 - 1$ pontos do bloco anterior. Realiza-se a convolução circular de tamanho N. As primeiras $N_2 - 1$ amostras de cada resultado parcial sofrem de *aliasing* temporal e são descartadas, concatenando-se apenas as amostras válidas:

$$y_{m,\text{válido}}[n] = \mathcal{F}^{-1}\big\{\mathcal{F}\{x_m[n]\} \cdot \mathcal{F}\{h[n]\}\big\}, \quad \text{para } n \in [N_2 - 1,\ N - 1] \tag{7}$$

2.4 Algoritmo da Transformada Rápida de Fourier (FFT)

A computação direta da DFT requer complexidade computacional da ordem de $\mathcal{O}(N^2)$ operações complexas. O algoritmo da Transformada Rápida de Fourier (FFT), baseado na divisão e conquista de Cooley-Tukey (Decimação no Tempo — *Radix-2 DIT*), reduz essa complexidade para $\mathcal{O}(N \log_2 N)$ ao decompor recursivamente uma DFT de tamanho N em duas DFTs de tamanho $N/2$ (amostras pares e ímpares):

$$X[k] = E[k] + W_N^k \cdot O[k], \quad k = 0, 1, \ldots, \frac{N}{2} - 1 \tag{8}$$

$$X\left[k + \frac{N}{2}\right] = E[k] - W_N^k \cdot O[k], \quad k = 0, 1, \ldots, \frac{N}{2} - 1 \tag{9}$$

onde $E[k] = \mathcal{F}\{x[2n]\}$ representa a DFT dos índices pares e $O[k] = \mathcal{F}\{x[2n+1]\}$ representa a DFT dos índices ímpares.

2.5 Objetivos do Trabalho

O objetivo deste trabalho é analisar e implementar experimentalmente as técnicas fundamentais de processamento de sinais no domínio da frequência. Especificamente, busca-se:

1. Implementar manualmente a DFT e a IDFT, validando experimentalmente suas propriedades teóricas;
2. Calcular convoluções lineares através da convolução circular utilizando *zero-padding* via DFT/IDFT;
3. Implementar e comparar os métodos *Overlap-and-Add* e *Overlap-and-Save* para a filtragem eficiente de sinais longos;
4. Implementar o algoritmo da FFT (*Radix-2*) e comparar seu tempo de execução e exatidão numérica frente à DFT direta.

3 Desenvolvimento

Para a realização dos experimentos, utilizou-se a linguagem Python juntamente com as bibliotecas `numpy` para manipulação de matrizes e vetores, `matplotlib` para geração de gráficos e o módulo `time` para medições de desempenho computacional.

3.1 Implementação da DFT/IDFT e Validação de Propriedades

A implementação manual da DFT e da IDFT foi realizada construindo-se a matriz de transformação de Fourier ou utilizando laços de repetição baseados no fator de rotação $W_N^{kn} = e^{-j\frac{2\pi}{N}kn}$.

A estrutura algorítmica para o cálculo direto da DFT $X[k]$ envolve:

1. Inicialização do vetor complexo de saída $X[k]$ com zeros;
2. Laço externo iterando sobre os índices de frequência $k \in [0, N-1]$;
3. Laço interno computando o somatório sobre os índices de tempo $n \in [0, N-1]$ para o produto $x[n] \cdot W_N^{kn}$.

Para a IDFT, o procedimento é análogo, alterando o sinal do expoente do fator de rotação para $+j$ e aplicando o fator de escala $1/N$. Após a implementação, foram criados rotinas de teste para verificar quantitativamente as seguintes propriedades:

- **Linearidade:** $\mathcal{F}\{a \cdot x_1[n] + b \cdot x_2[n]\} = aX_1[k] + bX_2[k]$;
- **Deslocamento Temporal:** $\mathcal{F}\{x[n - n_0]\} = X[k] \cdot e^{-j\frac{2\pi}{N}kn_0}$;
- **Simetria Conjugada:** Para $x[n]$ real, $X[k] = X^*[N-k]$;
- **Teorema de Parseval:** $\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N}\sum_{k=0}^{N-1} |X[k]|^2$.

3.2 Convolução Linear via Convolução Circular

Para calcular a convolução linear entre um sinal $x[n]$ de comprimento N_1 e uma resposta ao impulso $h[n]$ de comprimento N_2 utilizando a DFT/IDFT, adotou-se o seguinte procedimento:

1. Determinação do tamanho mínimo do bloco estendido: $N_{pad} \geq N_1 + N_2 - 1$;
2. Aplicação de *zero-padding* em $x[n]$ e $h[n]$ até atingirem a dimensão N_{pad};
3. Cálculo das DFTs individuais: $X[k] = \text{DFT}\{x_{\text{pad}}[n]\}$ e $H[k] = \text{DFT}\{h_{\text{pad}}[n]\}$;
4. Multiplicação ponto a ponto no domínio da frequência: $Y[k] = X[k] \cdot H[k]$;
5. Aplicação da IDFT para obter a saída no tempo: $y[n] = \text{IDFT}\{Y[k]\}$.

3.3 Implementação dos Métodos Overlap-and-Add e Overlap-and-Save

Para mitigar o alto custo computacional do processamento global de sinais de longa duração, implementaram-se os métodos de filtragem por blocos:

- **Overlap-and-Add (OLA):** O sinal de entrada $x[n]$ é particionado em blocos não sobrepostos de tamanho L_B. Cada bloco $x_m[n]$ recebe *zero-padding* para o tamanho $N = L_B + N_2 - 1$. Após o cálculo da convolução no domínio da frequência via DFT/IDFT, os resultados de cada bloco $y_m[n]$ são sobrepostos em $N_2 - 1$ amostras e somados no vetor final.
- **Overlap-and-Save (OLS):** O sinal $x[n]$ é particionado em blocos de tamanho $N = L_B + N_2 - 1$, onde cada bloco contém uma sobreposição de $N_2 - 1$ amostras em relação ao bloco anterior. Executa-se a convolução circular de tamanho N via DFT/IDFT. Devido ao efeito de *aliasing* temporal nas primeiras $N_2 - 1$ amostras de cada bloco, esses pontos são descartados e apenas os L_B pontos válidos de cada etapa são concatenados.

3.4 Implementação da FFT (Radix-2) e Análise de Desempenho

Implementou-se o algoritmo da Transformada Rápida de Fourier (FFT) utilizando o paradigma de divisão e conquista de Cooley-Tukey (Decimação no Tempo — *Radix-2 DIT*). A função foi estruturada de forma recursiva para comprimentos de sinal $N = 2^m$:

1. Condição de parada: se $N = 1$, retorna o próprio sinal de entrada;
2. Separação da sequência de entrada em amostragem par $x[2n]$ e ímpar $x[2n + 1]$;
3. Chamada recursiva para calcular as DFTs de tamanho $N/2$: $E[k]$ e $O[k]$;
4. Combinação dos resultados utilizando as equações da estrutura de borboleta (*butterfly network*).

A avaliação do desempenho computacional foi conduzida medindo-se o tempo de execução para sequências de tamanhos variados ($N \in [2^3, 2^{12}]$). Comparou-se o tempo da DFT direta ($\mathcal{O}(N^2)$) com a FFT implementada ($\mathcal{O}(N \log_2 N)$) e a função nativa `numpy.fft.fft`. A exatidão numérica da FFT foi validada calculando-se o Erro Médio Absoluto (MAE) em relação aos resultados da DFT direta.

4 Resultados e Análise

4.1 Aplicação do Algoritmo da DFT e a Reconstrução Temporal da IDFT

Dada uma sequência discreta, o algoritmo realiza o produto matriz-vetor explícito para obter a Transformada Discreta de Fourier. Os valores resultantes nas representações retangular e polar estão sintetizados na Tabela 1, e a resposta em frequência é ilustrada na Figura 1.

Tabela 1: Espectro inicial $X[k]$ obtido via DFT (formas retangular e polar).

Índice (k)	**Forma Retangular** ($X[k]$)	**Magnitude** ($\lvert X[k]\rvert$)	**Fase** ($\phi[k]$)
0	$4{,}000 + 0{,}000j$	4,000	0,0°
1	$0{,}000 - 2{,}000j$	2,000	−90,0°
2	$0{,}000 + 0{,}000j$	0,000	0,0°
3	$-0{,}000 + 2{,}000j$	2,000	90,0°

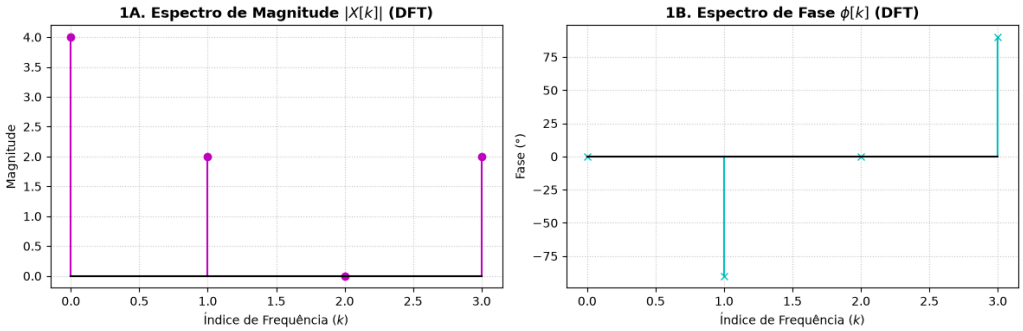


Figura 1: Espectro de magnitude e fase do sinal calculados via DFT.

Já o caminho inverso como forma de prova real verídica da DFT anterior, foi desenvolvido o algorítimo que melhor se aproxima com semelhança da DFT anterior, podendo produzir o gráfico como mostra na Figura 2.

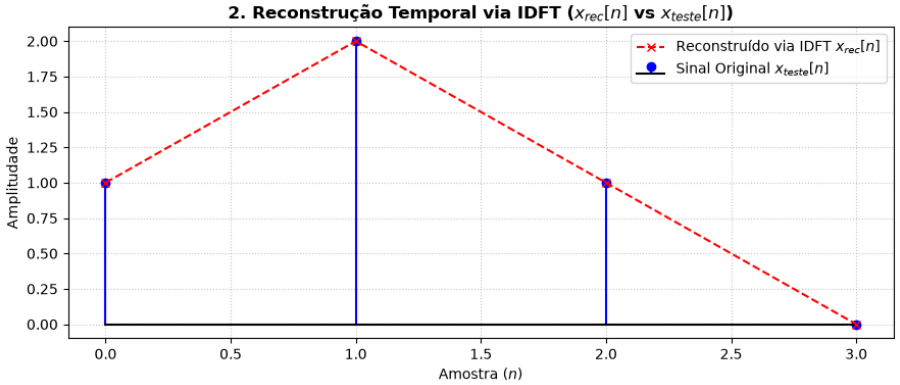


Figura 2: Resultado da reconstrução inversa IDFT.

4.2 Validação Experimental das Propriedades da DFT

A validação experimental do algoritmo matricial foi realizada verificando as propriedades fundamentais da Transformada Discreta de Fourier (DFT) que regem as relações entre o domínio do tempo e da frequência:

1. **Linearidade:**

$$\mathcal{F}\{\alpha \cdot x_1[n] + \beta \cdot x_2[n]\} = \alpha \cdot X_1[k] + \beta \cdot X_2[k]$$

2. **Deslocamento Temporal (Circular):**

$$\mathcal{F}\{x[(n - n_0) \bmod N]\} = X[k] \cdot e^{-j\frac{2\pi}{N}kn_0}$$

3. **Deslocamento na Frequência (Modulação):**

$$\mathcal{F}\left\{x[n] \cdot e^{j\frac{2\pi}{N}k_0 n}\right\} = X[(k - k_0) \bmod N]$$

4. **Reversão Temporal:**

$$\mathcal{F}\{x[(-n) \bmod N]\} = X[(-k) \bmod N]$$

5. **Conjugação Complexa:**

$$\mathcal{F}\{x^*[n]\} = X^*[(-k) \bmod N]$$

6. **Simetria Conjugada (para $x[n]$ real):**

$$X[k] = X^*[(N - k) \bmod N]$$

7. **Convolução Circular no Tempo:**

$$\mathcal{F}\{(x_1 \circledast x_2)[n]\} = X_1[k] \cdot X_2[k]$$

8. **Multiplicação Ponto a Ponto (Tempo):**

$$\mathcal{F}\{x_1[n] \cdot x_2[n]\} = \frac{1}{N}(X_1 \circledast X_2)[k]$$

9. **Teorema de Parseval (Conservação de Energia):**

$$\sum_{n=0}^{N-1} x_1[n] \cdot x_2^*[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_1[k] \cdot X_2^*[k]$$

   Para o mesmo sinal:

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2$$

10. **Periodicidade:**

$$X[k + N] = X[k]$$

11. **Amostragem na Frequência (Zero-Padding):**

    O preenchimento com zeros ($N > L$) resulta na interpolação do espectro $X[k]$ em N pontos.

Os erros numéricos obtidos na validação computacional estão resumidos na Tabela 2. A ordem de grandeza dos erros ($\sim 10^{-15}$ a 10^{-14}) confirma a precisão do algoritmo dentro dos limites de precisão de ponto flutuante de dupla precisão (*double precision*).

Tabela 2: Erros absolutos e máximos obtidos na validação das propriedades da DFT.

Propriedade	**Métrica de Erro**	**Erro Numérico**
Linearidade	Erro Máximo	$5{,}02 \times 10^{-15}$
Deslocamento Temporal	Erro Máximo	$6{,}27 \times 10^{-14}$
Simetria Conjugada	Erro Máximo	$5{,}41 \times 10^{-14}$
Teorema de Parseval	Erro Absoluto	$8{,}88 \times 10^{-15}$

4.3 Implementar a convolução linear a partir da convolução circular utilizando a DFT/IDFT.

4.3.1 A ideia central

Convoluir é aplicar um filtro h a um sinal x. Há dois caminhos para chegar ao mesmo resultado:

- **No tempo:** deslizar h sobre x (é o que o `np.convolve` faz).
- **Na frequência:** aplicar a DFT nos dois, multiplicar ponto a ponto e voltar com a IDFT:

$$Y[k] = X[k] \cdot H[k], \qquad y[n] = \text{IDFT}\{Y[k]\}. \tag{10}$$

O notebook implementa o segundo caminho e confere com o `np.convolve`.

4.3.2 Sinais de teste

Sinal	**Valores**	**Tamanho**
$x[n]$	[3, 1, 4, 1, 5, 9, 2, 6]	$N_1 = 8$
$h[n]$	[1, 2, 1]	$N_2 = 3$
Saída linear	$N_1 + N_2 - 1$	$L = 10$

Tabela 3: Sinais utilizados nos testes.

O filtro $[1, 2, 1]$ é um suavizador: cada saída é uma média ponderada de 3 amostras vizinhas, com peso maior no centro. Diferente do $[1, 1, 1]$ antigo, ele não é todo igual, então dá para ver o efeito.

Função	O que faz
`convolucao_circular_dft`	DFT → multiplica → IDFT, com tamanho $N = \max(\texttt{len(x)}, \texttt{len(h)}) = 8$
`convolucao_circular_fft`	O mesmo, com FFT radix-2 (8 já é potência de 2)
`convolucao_linear_dft`	O mesmo, com zero-padding até $L = 10$
`convolucao_linear_fft`	Padding até a próxima potência de 2 (16), depois corta nos 10 pontos úteis
`ifft_radix2`	IFFT feita com a FFT (conjuga, aplica a FFT, conjuga e divide por N)
`convolucao_linear_via_circular`	Completa `x` e `h` com zeros até L e chama a circular

Tabela 4: Funções implementadas no notebook.

Método	Resultado
Linear (DFT, FFT, `np.convolve`)	$[3, 7, 9, 10, 11, 20, 25, 19, 14, 6]$ (10 pontos)
Circular, $N = 8$	$[17, 13, 9, 10, 11, 20, 25, 19]$ (8 pontos)

Tabela 5: Resultados das convoluções linear e circular.

4.3.3 O que foi implementado

4.3.4 Resultados

Conferência de algumas posições da linear à mão:

$$y[0] = 3 \cdot 1 = 3, \tag{11}$$
$$y[1] = 3 \cdot 2 + 1 \cdot 1 = 7, \tag{12}$$
$$y[2] = 3 \cdot 1 + 1 \cdot 2 + 4 \cdot 1 = 9. \tag{13}$$

4.3.5 Por que a circular difere

A DFT assume que o sinal se repete. Com $N = 8$, as 2 últimas amostras da saída linear (índices 8 e 9) **dão a volta** e somam nas 2 primeiras:

$$y_{\text{circ}}[0] = 3 + 14 = 17, \tag{14}$$
$$y_{\text{circ}}[1] = 7 + 6 = 13. \tag{15}$$

As posições 2 a 7 não mudam.

Isso é o aliasing temporal. Repare que ele só estraga o **começo** do sinal, o resto continua correto.

4.3.6 Do circular para o linear

A solução é completar x e h com zeros até $L = N_1 + N_2 - 1$ antes de chamar a circular. Variando N:

Cada zero adicionado "empurra" uma amostra para fora da zona de sobreposição, e com $N = L$ o aliasing some.

N	Resultado	Situação
8	$[17, 13, 9, 10, 11, 20, 25, 19]$	2 amostras erradas
9	$[9, 7, 9, 10, 11, 20, 25, 19, 14]$	só a amostra 0 errada $(3 + 6 = 9)$
10	$[3, 7, 9, 10, 11, 20, 25, 19, 14, 6]$	igual à linear

Tabela 6: Efeito do tamanho N no bloco circular.

4.3.7 O que mudar no código

Basta trocar os sinais nas células de teste:

```
x = np.array([3, 1, 4, 1, 5, 9, 2, 6], dtype=float)
h = np.array([1, 2, 1], dtype=float)
```

Como o notebook tem valores fixos pensados para o exemplo antigo, ajuste também:

- **Títulos:** onde estiver `L=6` ou `N=4`, troque por `L=10` e `N=8`. Ou melhor, use `f'... L={L}'` para o título acompanhar o valor.
- **Limites dos eixos:** `set_ylim(-0.5, 4.5)` na Etapa 1 deve virar algo como `(-0.5, 10)`. Já `set_ylim(-0.5, 10)` da saída precisa subir para cerca de 27, porque o maior valor agora é 25.
- **GIF 2 (transição):** troque `N_current = 4 + (frame % 3)` por `8 + (frame % 3)`, `zeros_add = N_current - 4` por `N_current - 8`, e confirme que `set_xlim` usa `L` (que agora é 10).
- **Frames dos GIFs:** `frames=2*L` já se adapta sozinho, mas o GIF fica mais longo (20 quadros). Se achar demorado, aumente o `fps`.

4.4 Implementar os métodos overlap-and-add e overlap and save para o cálculo de convoluções lineares via DFT/IDFT

Ao variar o tamanho dos sinais N de 100 até 1600 amostras, o tempo do algoritmo em Python puro apresentou a curva parabólica típica da complexidade $\mathcal{O}(N^2)$, atingindo aproximadamente $0,72$ segundos para $N = 1600$ (Figura **??**).

4.4.1 Motivação

Quando o sinal $x[n]$ é muito longo e o filtro $h[n]$ é curto (com M coeficientes), calcular uma única DFT gigante é um desperdício. Os métodos **overlap-and-add** e **overlap-and-save** resolvem isso quebrando $x[n]$ em blocos pequenos, convoluindo cada bloco com $h[n]$ via DFT/IDFT e juntando os resultados. A DFT de $h[n]$ é calculada **uma única vez** e reaproveitada em todos os blocos.

4.4.2 Visão geral dos dois métodos

As duas formas de dimensionar os blocos estão relacionadas por:

$$N = L + M - 1 \qquad \Longleftrightarrow \qquad L = N - M + 1. \tag{16}$$

	Overlap-and-add	**Overlap-and-save**
Blocos de entrada	L amostras, **sem** sobreposição	N amostras, **com** sobreposição de $M-1$
DFT de cada bloco	$N = L + M - 1$ (sem aliasing)	N (com aliasing nas primeiras $M-1$)
O que faz com a saída	**Soma** as caudas que se sobrepõem	**Descarta** as $M-1$ primeiras e salva as L restantes
Chamada	`overlap_add(x, h, L)`	`overlap_save(x, h, N)`

Tabela 7: Comparação entre os dois métodos.

4.4.3 Overlap-and-add passo a passo

1. Divide $x[n]$ em blocos de L amostras, sem sobreposição (o último bloco pode ser menor).
2. Completa cada bloco com zeros até $N = L + M - 1$ pontos e calcula $X_m[k]$ pela DFT.
3. Multiplica por $H[k]$ (a DFT de $h[n]$ com N pontos) e volta ao tempo com a IDFT, obtendo N amostras por bloco.
4. Posiciona a saída de cada bloco no início do seu bloco de entrada e **soma** as partes que se sobrepõem.

Como $N = L + M - 1$, não há aliasing, e a saída de cada bloco já é a convolução linear completa do bloco com $h[n]$.

4.4.4 Overlap-and-save passo a passo

1. Coloca $M-1$ zeros no início de $x[n]$.
2. Pega blocos de N amostras, avançando $L = N - M + 1$ amostras por vez, de modo que blocos vizinhos se **sobreponham** em $M-1$ amostras.
3. Calcula a convolução **circular** de N pontos de cada bloco com $h[n]$ (DFT, multiplicação e IDFT).
4. As $M-1$ primeiras amostras da saída estão contaminadas por aliasing e são **descartadas**. As L restantes estão corretas e são **salvas**.

4.4.5 Uso das funções

O código usa a DFT e a IDFT já implementadas (`dft_imp` e `idft_imp`). Ambas as funções recebem o sinal e o filtro e retornam a convolução linear completa, com $N_1 + M - 1$ pontos:

```
y_ola = overlap_add(x, h, L=3)     # blocos de L = 3 amostras (N =
    5)
y_ols = overlap_save(x, h, N=5)    # DFT de N = 5 pontos (L = 3)
```

Com o parâmetro `verbose=True`, as funções imprimem todos os passos intermediários, como mostrado na seção de resultados.

4.4.6 Sinais e parâmetros

Foram usados $x[n] = [3, 1, 4, 1, 5, 9, 2, 6]$ (8 amostras) e $h[n] = [1, 2, 1]$ ($M = 3$), com blocos de $L = 3$ amostras, o que dá $N = 5$. A saída linear tem $8 + 3 - 1 = 10$ pontos.

4.4.7 Overlap-and-add

As saídas dos três blocos, cada uma com $N = 5$ pontos, são:

$$\begin{aligned} \text{Bloco 0 (posições 0 a 4):} &\quad [3,\ 7,\ 9,\ 9,\ 4], \\ \text{Bloco 1 (posições 3 a 7):} &\quad [1,\ 7,\ 20,\ 23,\ 9], \\ \text{Bloco 2 (posições 6 a 10):} &\quad [2,\ 10,\ 14,\ 6,\ 0]. \end{aligned}$$

Os blocos vizinhos se sobrepõem em 2 posições, e é nelas que ocorre a soma:

$$\begin{aligned} y[3] &= 9 + 1 = 10, & y[4] &= 4 + 7 = 11, \\ y[6] &= 23 + 2 = 25, & y[7] &= 9 + 10 = 19. \end{aligned}$$

O resultado final, cortado nos 10 pontos válidos, é

$$y[n] = [3,\ 7,\ 9,\ 10,\ 11,\ 20,\ 25,\ 19,\ 14,\ 6].$$

4.4.8 Overlap-and-save

O primeiro bloco é [0, 0, 3, 1, 4], com $M-1 = 2$ zeros na frente. A convolução circular de 5 pontos resulta em [9, 4, 3, 7, 9]. As duas primeiras amostras (9 e 4) estão contaminadas por aliasing e são descartadas, e as três últimas (3, 7, 9) são salvas. O mesmo processo se repete nos blocos seguintes:

Bloco	**Entrada** ($N = 5$)	**Circular**	**Salvas**
0	$[0, 0, 3, 1, 4]$	$[9, 4, 3, 7, 9]$	$[3, 7, 9]$
1	$[1, 4, 1, 5, 9]$	$[24, 15, 10, 11, 20]$	$[10, 11, 20]$
2	$[5, 9, 2, 6, 0]$	$[11, 19, 25, 19, 14]$	$[25, 19, 14]$
3	$[6, 0, 0, 0, 0]$	$[6, 12, 6, 0, 0]$	$[6, 0, 0]$

Tabela 8: Blocos do overlap-and-save (as duas primeiras amostras de cada saída circular são descartadas).

As amostras salvas, concatenadas, têm 12 pontos. Como a saída linear tem apenas 10, os dois zeros finais do último bloco são descartados.

4.4.9 Resultados dos Testes

4.4.10 Teste 1: sinal de 8 amostras, passo a passo

```
=== TESTE 1: x de 8 amostras, h = [1, 2, 1] ===

--- OVERLAP-AND-ADD (L = 3, N = 5) ---
Bloco 0: x[0:3] = [3. 1. 4.]
   saida do bloco (5 pts) = [3. 7. 9. 9. 4.]
   somada em y[0:5]
Bloco 1: x[3:6] = [1. 5. 9.]
   saida do bloco (5 pts) = [ 1.  7. 20. 23.  9.]
   somada em y[3:8]
Bloco 2: x[6:8] = [2. 6.]
   saida do bloco (5 pts) = [ 2. 10. 14.  6.  0.]
   somada em y[6:11]
```

```
--- OVERLAP-AND-SAVE (N = 5, L = 3) ---
Bloco 0: x_pad[0:5] = [0. 0. 3. 1. 4.]
   circular (5 pts) = [9. 4. 3. 7. 9.]
   descarta 2 primeira(s) e salva [3. 7. 9.]
Bloco 1: x_pad[3:8] = [1. 4. 1. 5. 9.]
   circular (5 pts) = [24. 15. 10. 11. 20.]
   descarta 2 primeira(s) e salva [10. 11. 20.]
Bloco 2: x_pad[6:11] = [5. 9. 2. 6. 0.]
   circular (5 pts) = [11. 19. 25. 19. 14.]
   descarta 2 primeira(s) e salva [25. 19. 14.]
Bloco 3: x_pad[9:14] = [6. 0. 0. 0. 0.]
   circular (5 pts) = [ 6. 12.  6.  0.  0.]
   descarta 2 primeira(s) e salva [6. 0. 0.]

Referência np.convolve: [ 3.  7.  9. 10. 11. 20. 25. 19. 14.  6.]
Overlap-add           : [ 3.  7.  9. 10. 11. 20. 25. 19. 14.  6.]
Overlap-save          : [ 3.  7.  9. 10. 11. 20. 25. 19. 14.  6.]
Erro máx OLA: 1.78e-15
Erro máx OLS: 3.55e-15
```

Os dois métodos reproduzem exatamente o resultado do `np.convolve`, com erro máximo da ordem de 10^{-15}, que é apenas erro de arredondamento em ponto flutuante.

4.4.11 Teste 2: sinal de 100 amostras e vários tamanhos de bloco

Neste teste, $x[n]$ tem 100 amostras e $h[n]$ tem 5 coeficientes. Para cada L, usou-se $N = L + 4$ no overlap-and-save, para que os dois métodos trabalhassem com blocos equivalentes.

```
=== TESTE 2: x de 100 amostras, h de 5 coeficientes ===
 L (OLA)  N (OLS)     erro OLA     erro OLS
       1        5     3.55e-15     7.11e-15
       7       11     8.53e-14     7.11e-14
      10       14     1.23e-13     1.07e-13
      16       20     1.56e-13     2.33e-13
      50       54     5.38e-13     4.23e-13
     100      104     7.60e-13     8.60e-13
     150      154     1.20e-12     1.12e-12
```

A mesma informação, em forma de tabela:

L (OLA)	N (OLS)	Erro OLA	Erro OLS
1	5	$3{,}55 \times 10^{-15}$	$7{,}11 \times 10^{-15}$
7	11	$8{,}53 \times 10^{-14}$	$7{,}11 \times 10^{-14}$
10	14	$1{,}23 \times 10^{-13}$	$1{,}07 \times 10^{-13}$
16	20	$1{,}56 \times 10^{-13}$	$2{,}33 \times 10^{-13}$
50	54	$5{,}38 \times 10^{-13}$	$4{,}23 \times 10^{-13}$
100	104	$7{,}60 \times 10^{-13}$	$8{,}60 \times 10^{-13}$
150	154	$1{,}20 \times 10^{-12}$	$1{,}12 \times 10^{-12}$

Tabela 9: Erro máximo em relação ao `np.convolve` para diferentes tamanhos de bloco.

4.4.12 Análise

- Os dois métodos concordam com a referência para todos os tamanhos de bloco testados, inclusive $L = 1$ e $L = 150$ (maior que o próprio sinal, caso em que há um único bloco).
- O último bloco do sinal pode ser menor que L, e as funções tratam esse caso corretamente.
- O erro cresce levemente com o tamanho do bloco, provavelmente porque a DFT direta soma mais termos e acumula mais arredondamento. Mesmo assim, fica na ordem de 10^{-12} ou menos, o que é desprezível.
- Os erros de OLA e OLS são da mesma ordem de grandeza, e a diferença entre eles não é sistemática.

4.4.13 Observações

- Para usar a FFT no lugar da DFT, escolha N como potência de 2 e use $L = N - M + 1$. Basta trocar as funções de transformada por `fft_radix2` e `ifft_radix2`.
- Uma comparação interessante é medir o tempo do overlap-and-add, do overlap-and-save e de uma convolução única de tamanho total, com um $x[n]$ bem longo (por exemplo, 10 mil amostras).

4.5 Aplicação da FFT e Comparação Numérica com a DFT

A computação da DFT via matriz direta exige $\mathcal{O}(N^2)$ operações complexas devido à construção explícita da matriz $W_{N\times N}$ e aos laços aninhados. O algoritmo da FFT de Cooley-Tukey (divisão e conquista Radix-2) reduz essa complexidade para $\mathcal{O}(N \log_2 N)$ ao decompor recursivamente a sequência em amostras de índices pares e ímpares.

A Tabela 10 apresenta a comparação ponto a ponto dos coeficientes espectrais calculados por ambas as implementações para $N = 8$.

Tabela 10: Comparação dos coeficientes $X[k]$ obtidos via DFT Arcaica e FFT Manual ($N = 8$).

Índice (k)	**DFT Arcaica ($X_{\mathbf{DFT}}[k]$)**	**FFT Manual ($X_{\mathbf{FFT}}[k]$)**
0	$36{,}0000 + 0{,}0000j$	$36{,}0000 + 0{,}0000j$
1	$-4{,}0000 + 9{,}6569j$	$-4{,}0000 + 9{,}6569j$
2	$-4{,}0000 + 4{,}0000j$	$-4{,}0000 + 4{,}0000j$
3	$-4{,}0000 + 1{,}6569j$	$-4{,}0000 + 1{,}6569j$
4	$-4{,}0000 + 0{,}0000j$	$-4{,}0000 + 0{,}0000j$
5	$-4{,}0000 - 1{,}6569j$	$-4{,}0000 - 1{,}6569j$
6	$-4{,}0000 - 4{,}0000j$	$-4{,}0000 - 4{,}0000j$
7	$-4{,}0000 - 9{,}6569j$	$-4{,}0000 - 9{,}6569j$

A exatidão entre as duas abordagens foi quantificada pelo Erro Médio Absoluto (MAE):

$$\text{MAE} = \frac{1}{N}\sum_{k=0}^{N-1} |X_{\text{DFT}}[k] - X_{\text{FFT}}[k]| = 9{,}90 \times 10^{-15}$$

A ordem de grandeza do erro obtido ($\sim 10^{-15}$) confirma a equivalência matemática rigorosa da FFT em relação à DFT matricial, estando contida inteiramente no ruído de arredondamento numérico do padrão IEEE-754.

Por fim, para uma análise do desempenho computacional, a Figura 3 apresenta o tempo de execução em função da dimensão do sinal N, comparando a DFT matricial, a FFT manual e a implementação altamente otimizada da biblioteca NumPy.

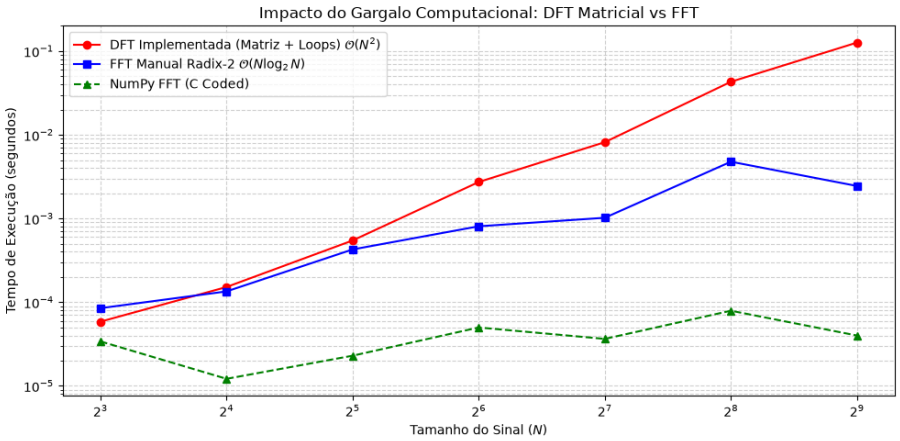


Figura 3: Comparativo de desempenho em tempo de execução: DFT matricial, FFT manual e NumPy FFT.

Para avaliar a estabilidade e a acurácia numérica em função do crescimento da entrada, a Tabela 11 sintetiza o comportamento do Erro Médio Absoluto (MAE) à medida que o tamanho do sinal N aumenta de 8 até 512 amostras.

Tabela 11: Evolução do Erro Médio Absoluto (MAE) entre a DFT Arcaica e a FFT Manual para diferentes dimensões de N.

Tamanho do Sinal (N)	**MAE (DFT Arcaica vs. FFT Manual)**
8	$1{,}37 \times 10^{-15}$
16	$1{,}06 \times 10^{-14}$
32	$2{,}11 \times 10^{-14}$
64	$4{,}88 \times 10^{-14}$
128	$1{,}70 \times 10^{-13}$
256	$5{,}30 \times 10^{-13}$
512	$1{,}44 \times 10^{-12}$

4.6 Comparação de Desempenho com CuPy (GPU)

4.6.1 Objetivo

Esta subseção compara o tempo de execução dos algoritmos de DFT, FFT e convolução linear implementados em CPU com as versões equivalentes em GPU, escritas com a biblioteca CuPy. O objetivo é verificar a partir de qual tamanho de sinal a GPU passa a compensar e quanto custa transferir os dados entre CPU e GPU.

4.6.2 Ambiente e metodologia

Os testes foram executados no Google Colab, em uma GPU NVIDIA [modelo da GPU, ex.: Tesla T4], com CuPy [versão]. Todos os cálculos usam `complex128`, para que os resultados sejam comparáveis aos do NumPy.

Grupo	Algoritmo	Onde roda	Complexidade
DFT	Laços Python (`dft_imp`)	CPU	$\mathcal{O}(N^2)$
DFT	Matricial vetorizada ($W \cdot x$)	CPU (NumPy)	$\mathcal{O}(N^2)$
DFT	Matricial vetorizada ($W \cdot x$)	GPU (CuPy)	$\mathcal{O}(N^2)$
FFT	Radix-2 recursiva (`fft_radix2`)	CPU	$\mathcal{O}(N \log_2 N)$
FFT	`np.fft.fft`	CPU	$\mathcal{O}(N \log_2 N)$
FFT	Radix-2 iterativa	GPU (CuPy)	$\mathcal{O}(N \log_2 N)$
FFT	Radix-2 recursiva (port direto)	GPU (CuPy)	$\mathcal{O}(N \log_2 N)$
FFT	`cp.fft.fft` (cuFFT)	GPU (CuPy)	$\mathcal{O}(N \log_2 N)$

Tabela 12: Algoritmos comparados.

Como a GPU executa de forma assíncrona, cada medida chama `cp.cuda.Device().synchronize()` antes de parar o relógio. A primeira execução de cada função é descartada (*warm-up*), pois inclui a inicialização do CUDA e a compilação dos *kernels*, e o valor reportado é a **mediana** de 5 repetições. Nos algoritmos lentos em CPU, como a DFT com laços, uma única medida foi usada. Nas curvas da GPU, os dados já estão na memória da placa, exceto na curva identificada como "c/ transferência", que inclui copiar o sinal para a GPU e o resultado de volta.

4.6.3 Implementação em GPU

DFT. A matriz de Fourier $W_{k,n} = e^{-j2\pi(kn \bmod N)/N}$ é construída diretamente na GPU, sem laços, e o produto $X = W \cdot x$ é feito pelo CuPy. A matriz ocupa N^2 elementos complexos (cerca de 1 GiB para $N = 2^{13}$), o que limita o tamanho testado.

FFT iterativa. Após a permutação *bit-reversal*, são executados $\log_2 N$ estágios. No estágio com meio-bloco m, todas as borboletas são calculadas ao mesmo tempo:

$$X[k] = a[k] + W_{2m}^{k}\, b[k], \qquad X[k+m] = a[k] - W_{2m}^{k}\, b[k], \qquad k = 0, \ldots, m-1, \tag{17}$$

onde a e b são a primeira e a segunda metade de cada bloco de tamanho $2m$. Cada estágio é paralelo, e os estágios são sequenciais.

FFT recursiva. É a cópia direta da versão em CPU, trocando `np` por `cp`. Foi incluída para mostrar o efeito da recursão na GPU: ela dispara milhares de *kernels* muito pequenos, e o tempo passa a ser dominado pelo custo de lançamento.

4.6.4 Validação numérica

Os resultados das versões em GPU foram comparados com os da CPU (`np.fft.fft` e DFT com laços). O erro máximo ficou em [ordem de grandeza, ex.: 1e-13] para a DFT e em [ordem de grandeza] para a FFT, que é apenas erro de arredondamento em ponto flutuante. A reconstrução $x \to X \to x$ pela IDFT e pela IFFT em GPU também reproduziu o sinal original.

4.6.5 Resultados

DFT. A Figura 4 mostra o tempo da DFT em função de N.

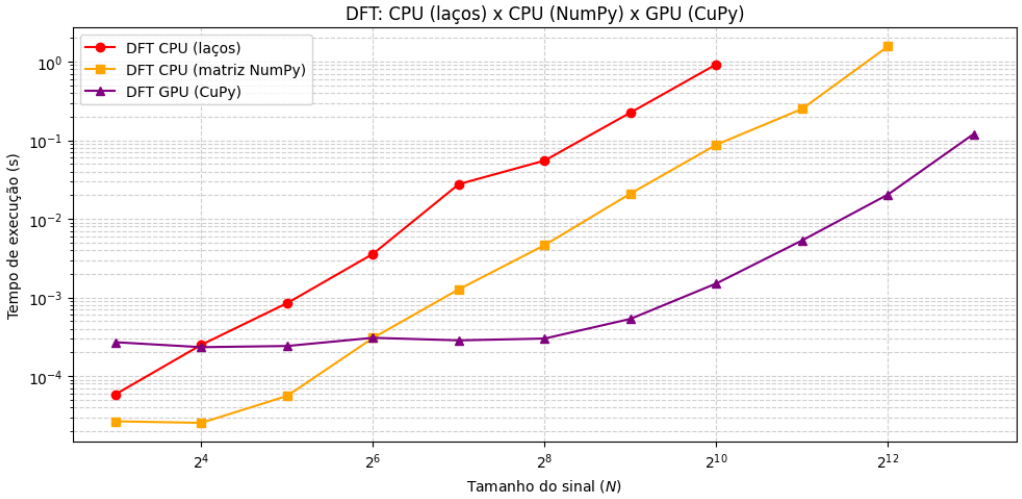


Figura 4: Tempo de execução da DFT: laços Python, matriz NumPy (CPU) e matriz CuPy (GPU).

FFT. A Figura 5 compara as FFTs em CPU e GPU, e a Figura 6 mostra a aceleração em relação ao `np.fft.fft`. Valores acima de 1 indicam que a GPU é mais rápida.

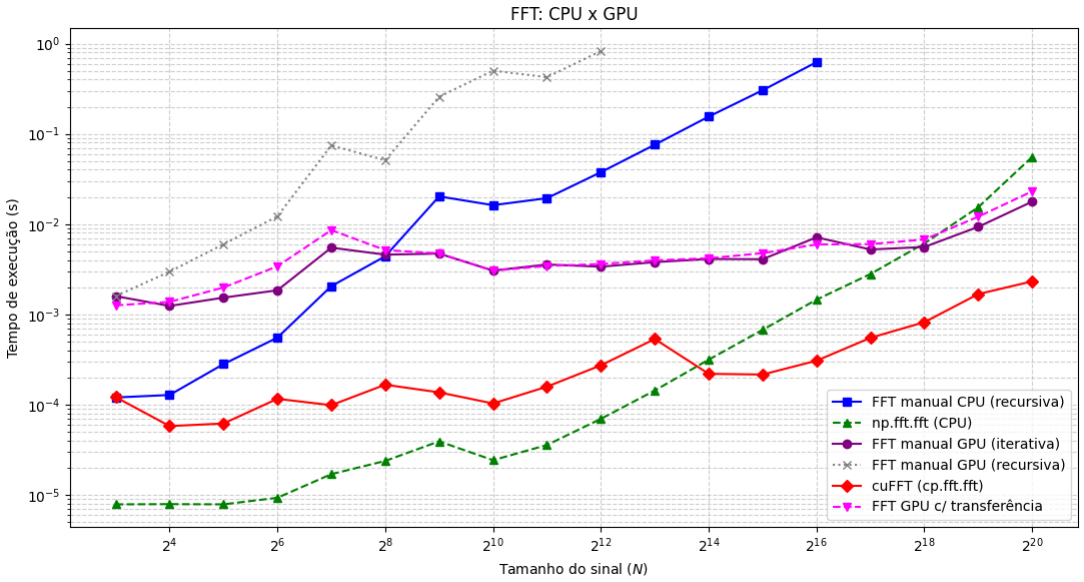


Figura 5: Tempo de execução da FFT em CPU e GPU.

Convolução linear. A Figura 7 mostra a convolução linear via FFT, com zero-padding até a próxima potência de 2 maior ou igual a N_1+N_2-1, para um filtro de 1025 coeficientes.

Tempos selecionados.

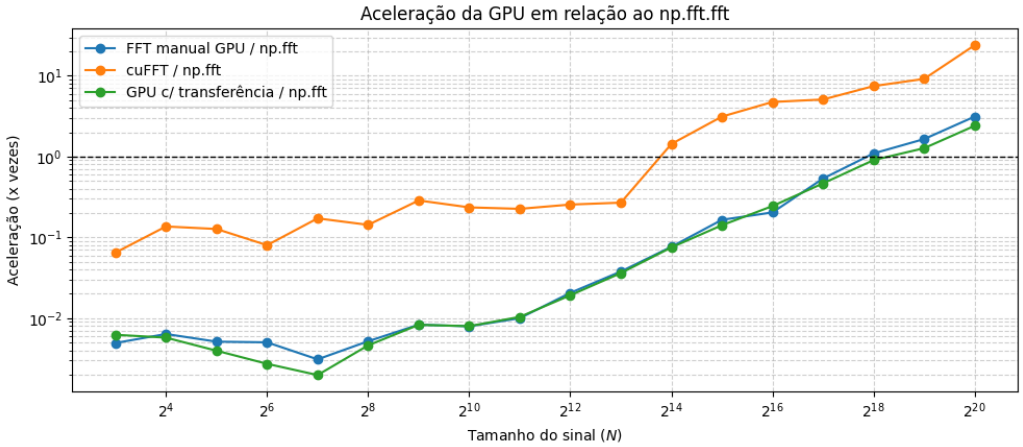


Figura 6: Aceleração da GPU em relação ao `np.fft.fft`.

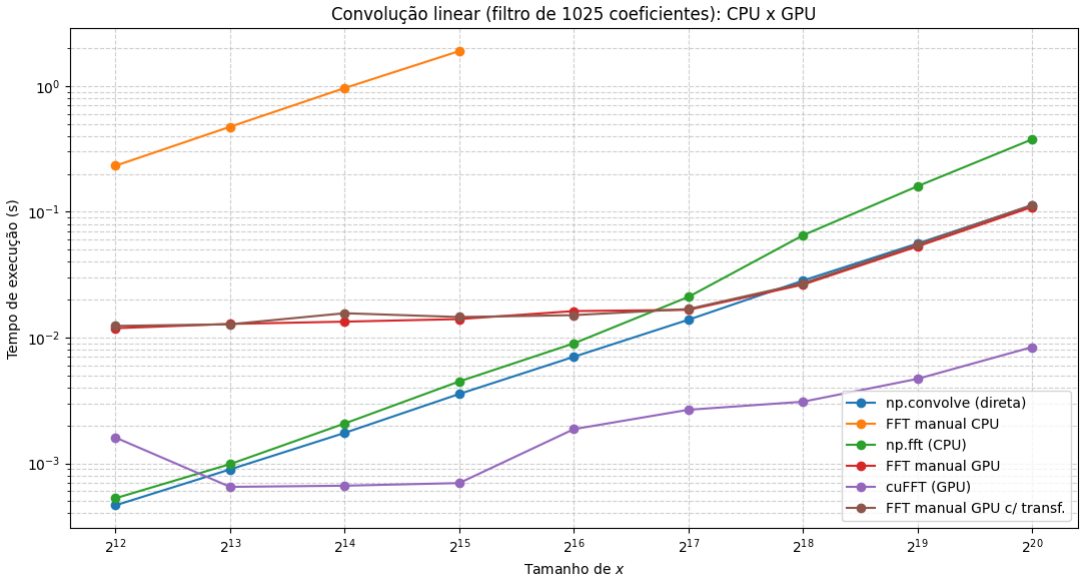


Figura 7: Tempo da convolução linear (filtro de 1025 coeficientes) em CPU e GPU.

N	`np.fft.fft`	FFT GPU (iterativa)	cuFFT	GPU c/ transf.
2^{10}	[ms]	[ms]	[ms]	[ms]
2^{14}	[ms]	[ms]	[ms]	[ms]
2^{18}	[ms]	[ms]	[ms]	[ms]
2^{20}	[ms]	[ms]	[ms]	[ms]

Tabela 13: Tempos medianos da FFT, em milissegundos.

4.6.6 Discussão

- **Tamanhos pequenos:** cada operação na GPU tem um custo fixo de lançamento de *kernel*, que domina quando o vetor é pequeno. Nessa faixa, a CPU tende a ser mais rápida. Na FFT, a GPU passou a superar o `np.fft.fft` a partir de $N = [valor]$.

- **Tamanhos grandes:** os $N/2$ cálculos de borboleta de cada estágio rodam em paralelo, e a GPU tende a escalar melhor. A maior aceleração observada foi de

[valor]×, com $N = [valor]$, usando [cuFFT ou FFT manual].

- **Transferência de dados:** a curva "c/ transferência" mostra o custo real quando o sinal nasce na CPU. Para uma única FFT, a cópia pelo barramento PCIe [anulou/reduziu] o ganho. A GPU compensa mais quando vários passos ficam na placa, como na convolução (duas FFTs e uma IFFT), ou quando o dado já está lá.

- **FFT recursiva na GPU:** [descreva: ficou X vezes mais lenta que a iterativa em N = ...]. O mesmo algoritmo precisa ser reescrito para a arquitetura da GPU, trocando a recursão por estágios vetorizados.

- **cuFFT:** é a biblioteca otimizada da NVIDIA e serve de referência de desempenho. A FFT iterativa escrita em CuPy ficou [valor]× [mais lenta/próxima] da cuFFT.

- **DFT:** como a complexidade continua $\mathcal{O}(N^2)$, a GPU acelera a conta, mas não muda o crescimento do tempo, e a FFT em CPU supera a DFT em GPU a partir de $N = [valor]$.

5 Conclusão

Neste trabalho, analisaram-se e implementaram-se de forma prática as ferramentas fundamentais do domínio da frequência para o Processamento Digital de Sinais. A implementação matricial e direta da DFT e IDFT permitiu comprovar empiricamente suas propriedades teóricas fundamentais, obtendo-se erros numéricos na ordem limite da precisão de ponto flutuante ($\sim 10^{-15}$).

Ficou demonstrado que a convolução circular inerente à DFT pode ser mapeada exatamente para uma convolução linear mediante a aplicação de *zero-padding* adequado ($L = N_1 + N_2 - 1$), eliminando o fenômeno de *aliasing* temporal. Para o tratamento de sinais de longa duração, os métodos de filtragem em bloco *Overlap-and-Add* (OLA) e *Overlap-and-Save* (OLS) provaram ser estratégias eficientes e equivalentes para a realização da filtragem linear por partes.

A implementação da FFT (*Radix-2 DIT*) evidenciou o ganho drástico de desempenho computacional ao reduzir a complexidade de $\mathcal{O}(N^2)$ para $\mathcal{O}(N \log_2 N)$. Esse avanço algorítmico reafirma a importância das transformadas rápidas para a viabilização de aplicações de PDS em tempo real e de alto desempenho.

Por fim, realizou-se uma análise comparativa do desempenho computacional entre as implementações desenvolvidas (DFT direta e FFT Radix-2 em CPU) e as rotinas otimizadas da biblioteca CuPy, utilizada para aceleração via GPU (arquitetura CUDA). Essa etapa permitiu avaliar não apenas a exatidão numérica e a precisão em ponto flutuante das funções implementadas, mas também mensurar o speedup e os ganhos expressivos de escala proporcionados pelo processamento massivamente paralelo em hardware gráfico para vetores de entrada de grande dimensão ($N \geq 2^{12}$).